



Architettura degli elaboratori

Lezione 10 - teoria

Prof.ssa Valentina Ciriani

Università degli Studi di Milano

www.di.unimi.it/ciriani

1



Contenuto

- Progetto CPU a singolo ciclo [PH 4.4]
Unità di controllo

-MKM = M. Morris Mano, C.R. Kime, T. Martin, Reti logiche, Pearson

-PH = D.A. Patterson, J.L. Hennessy, Struttura e Progetto dei Calcolatori, Zanichelli

2

2



Unità di controllo

- Abbiamo costruito **un'unità di elaborazione** a ciclo singolo
- Vediamo ora **l'unità di controllo** che invia i segnali di controllo alla unità di elaborazione ed è formata di due parti:
 1. Unità di **Controllo della ALU**, che genera i segnali di controllo per la ALU
 2. l'Unità di **Controllo principale**, che in funzione del codice operativo dell'istruzione da eseguire, genera:
 1. i vari segnali di controllo (mux, registri, memoria, ecc.)
 2. gli input per l'unità di controllo della ALU

3

3



Unità di controllo per la ALU

Ha due input:

1. Il segnale **ALUOp** che è generato dalla l'Unità di Controllo principale (utilizzando il capo op dell'istruzione)
2. Il campo func delle istruzioni di tipo R

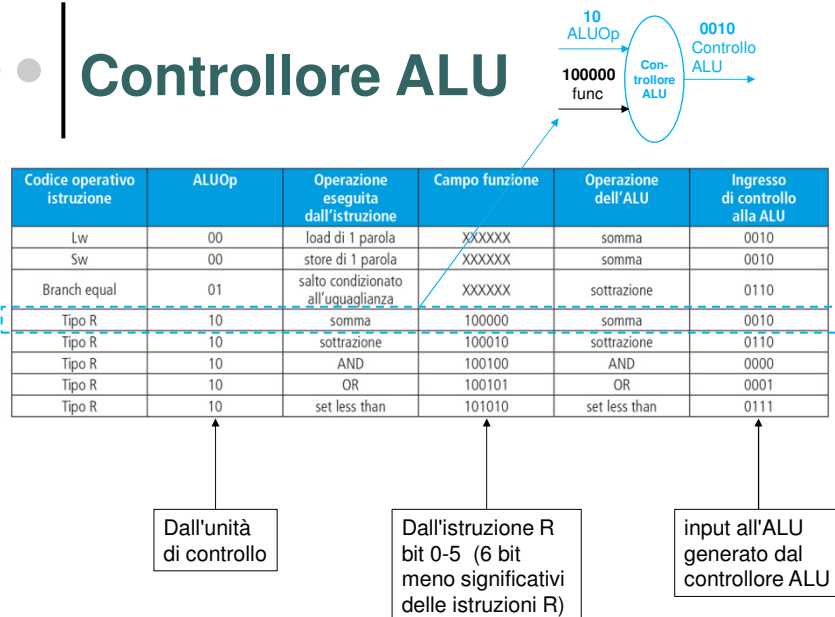
Il segnale **ALUOp** di 2 bit, ha il seguente significato:

1. **ALUOp=00** Somma per load e store
2. **ALUOp=01** Sottrazione per beq
3. **ALUOp=10** Operazione aritmetico-logica specificata dal campo funct dell'istruzione di tipo R

4

4

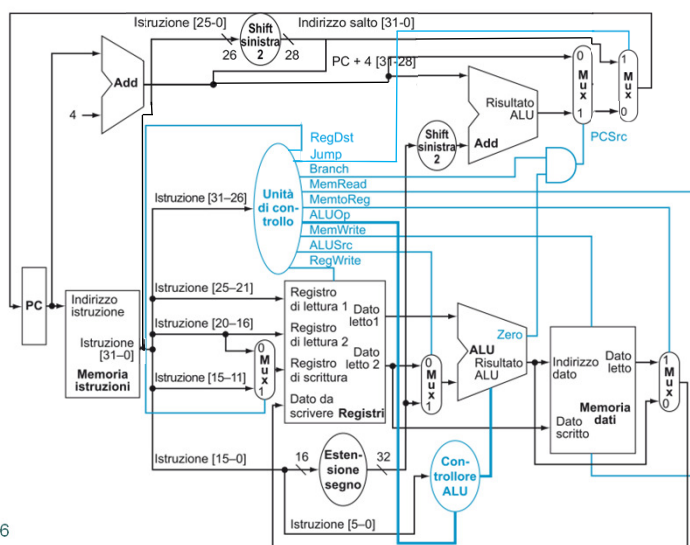
Controllore ALU



5

5

Schema controllo completo



6

6



Unità di controllo

Bisogna definire i 10 bit di controllo per

- 5 multiplexer:
 - **RegDst** (se il registro destinazione è il secondo **0** o il terzo **1** nell'istruzione)
 - **ALUSrc** (se il secondo operatore ALU viene dai registri **0** o dall'estensione del segno **1**)
 - **MemtoReg** (se il dato da scrivere nel registro viene dalla ALU **0** o dalla memoria **1**)
 - **Branch** (usato in AND con Zero, se il calcolo del PC è PC+4 **0** o viene dalla ALU **1**)
 - **Jump** (se il calcolo del PC viene direttamente dall'istruzione di jump **1** o meno **0**)
- Abilitazione alla scrittura nel Banco dei Registri
 - **RegWrite** (se si deve scrivere su un registro **1** o meno **0**)
- Accesso in lettura alla memoria dati
 - **MemRead** (se si deve leggere dalla memoria **1** o meno **0**)
- Accesso in scrittura alla memoria dati
 - **MemWrite** (se si deve scrivere in memoria **1** o meno **0**)
- Segnali al controllore ALU
 - **ALUOp** (2 bit descritti prima, che definiscono il tipo di operazione da fare)

7

7



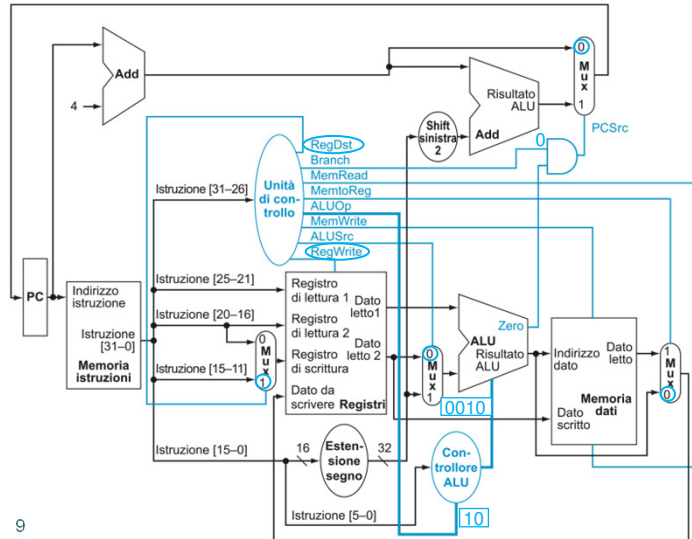
Unità di controllo

		Istruzione (op)				
Nome del segnale		Formato R (000000)	lw (100011)	sw (101011)	beq (000100)	j (000010)
mux	RegDst	1	0	-	-	-
	ALUSrc	0	1	1	0	-
	MemtoReg	0	1	-	-	-
	Branch	0	0	0	1	-
	Jump	0	0	0	0	1
registri	RegWrite	1	1	0	0	0
memoria	MemRead	0	1	0	0	0
	MemWrite	0	0	1	0	0
ALU	ALUOp	10	00	00	01	-

8

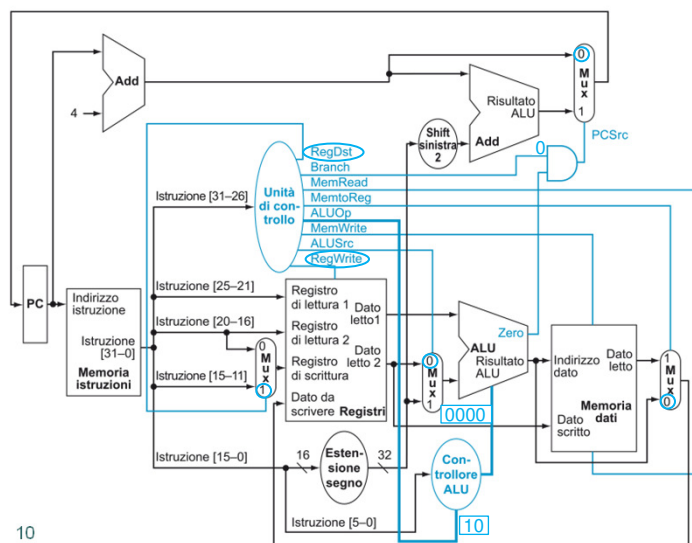
8

● ● ● | Esempio add



9

Esempio and



10

[illegible]

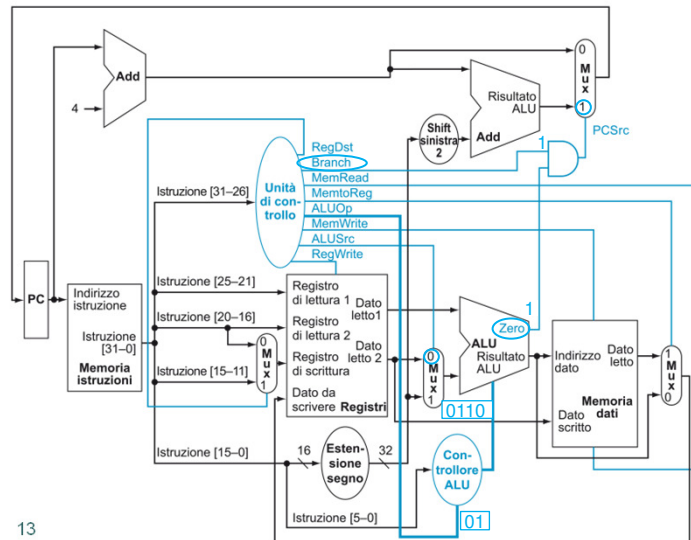
12

[illegible]

12



Esempio beq con uguale

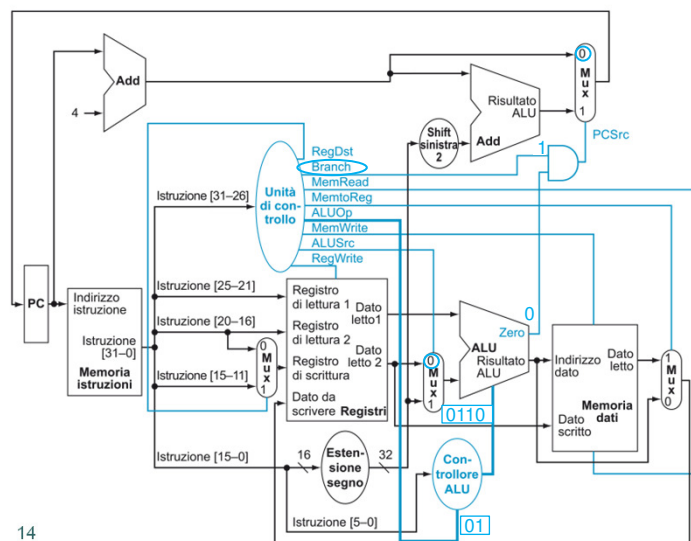


13

13



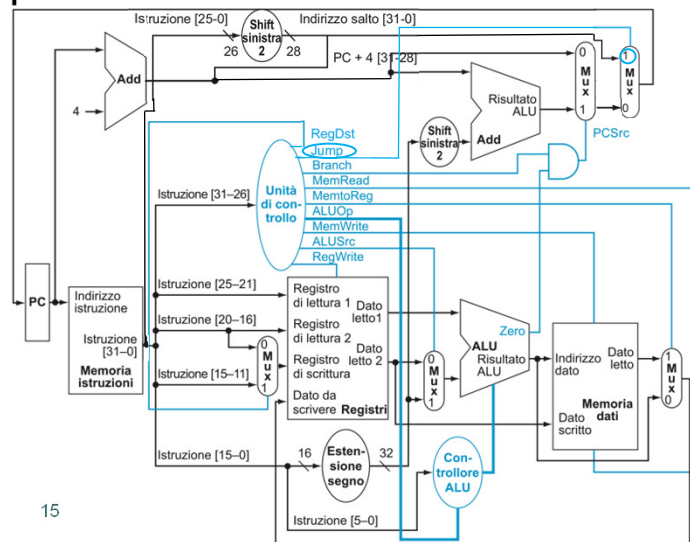
Esempio beq con diverso



14

14

Esempio j



15

Ciclo singolo ed efficienza

- Ogni istruzione viene iniziata e completata in un ciclo di clock
- Il ciclo di clock deve avere la stessa lunghezza per ogni istruzione
- La lunghezza del ciclo di clock è determinata dal percorso più lungo (istruzione con esecuzione più lenta)

Esempio

`lw` utilizza 5 unità in serie:

1. Lettura istruzione
2. Lettura registri
3. Calcolo ALU
4. Accesso memoria
5. Scrittura registro

`beq` utilizza 3 unità in serie:

1. Lettura istruzione
2. Lettura registri
3. Calcolo ALU

Quindi alcune istruzioni potrebbero essere implementate con un ciclo di clock più breve

16

16



CPU multi-ciclo

Ogni **fase** d'esecuzione di un'istruzione (e non una istruzione intera) richiede un ciclo di clock

Vantaggi principali:

- Si può usare un ciclo di clock più piccolo
- Il vincolo sul clock è dato dalla fase più lunga (e non più dall'istruzione più lunga)
- Aumento della velocità di esecuzione
 - non in termini di velocità della singola istruzione,
 - ma in termini di **throughput** (numero di istruzioni per unità di tempo)

Costo aggiuntivo:

- Necessario introdurre dei registri interni aggiuntivi per memorizzare i valori da usare nei cicli di clock successivi
- Progettazione mediante **pipeline** (sovrapposizione temporale dell'esecuzione delle varie istruzioni)

17

17



Architettura degli elaboratori

Lezione 10 - esercizi

Prof.ssa Valentina Ciriani

Università degli Studi di Milano

www.di.unimi.it/ciriani

18



Esercizi

Si assuma che

- o \$s1 contenga 0x4
- o \$s2 contenga 4
- o \$s3 contenga 0x2A
- o \$s4 contenga 0x30000018
- o PC contenga 0x53440040 (indirizzo istruzione)
- o L1 sia 2
- o una parte di memoria dati sia

0x30000010	0x00000000
0x30000014	0x00000000
0x30000018	0x00000001
0x3000001C	0x00000002
0x3000001E	0x00000003

Descrivere l'esecuzione delle seguenti istruzioni (usare lo schema per R e I)

1. `add $s5, $s1, $s2`
2. `sub $s5, $s1, $s2`
3. `and $s5, $s1, $s2`
4. `or $s5, $s1, $s2`
5. `slt $s5, $s1, $s2`
6. `lw $s1, 4($s4)`
7. `sw $s5, 4($s4)`
8. `beq $s1, $s2, L1`
9. `beq $s1, $s3, L1`

Descrivere l'esecuzione della seguente istruzione (usare lo schema per J)

10. `j 32`

19

19

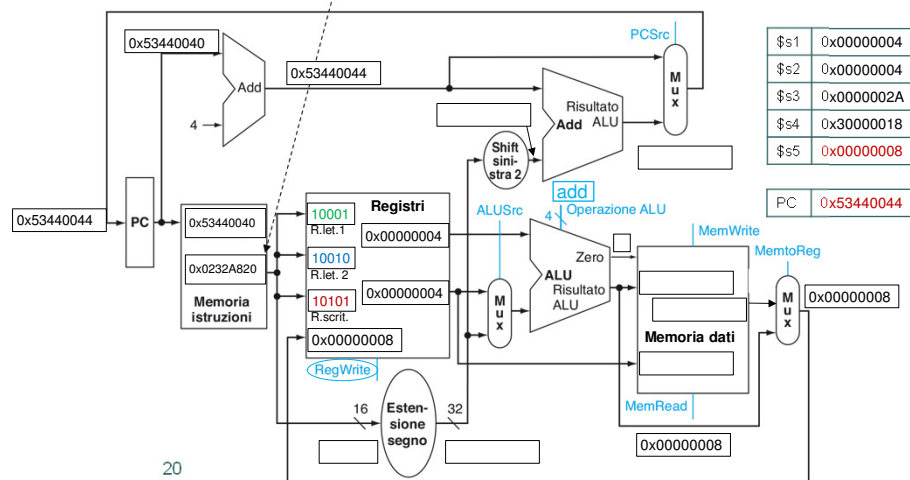


Esercizio 1

PC 0x53440040

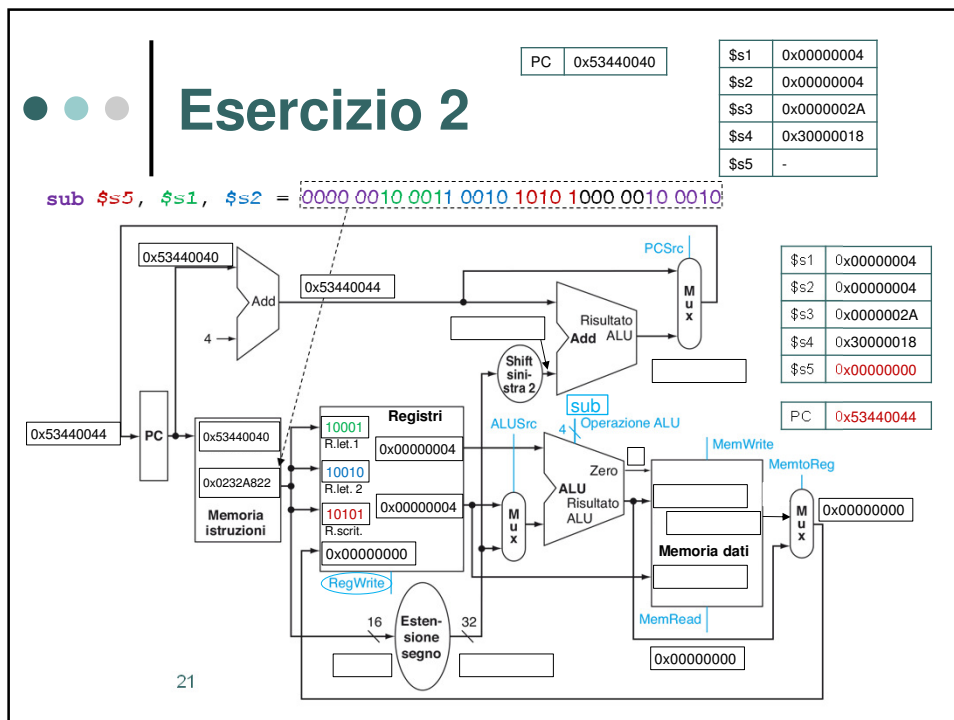
\$s1	0x00000004
\$s2	0x00000004
\$s3	0x0000002A
\$s4	0x30000018
\$s5	-

`add $s5, $s1, $s2` = 0000 0010 0011 0010 1010 1000 0010 0000

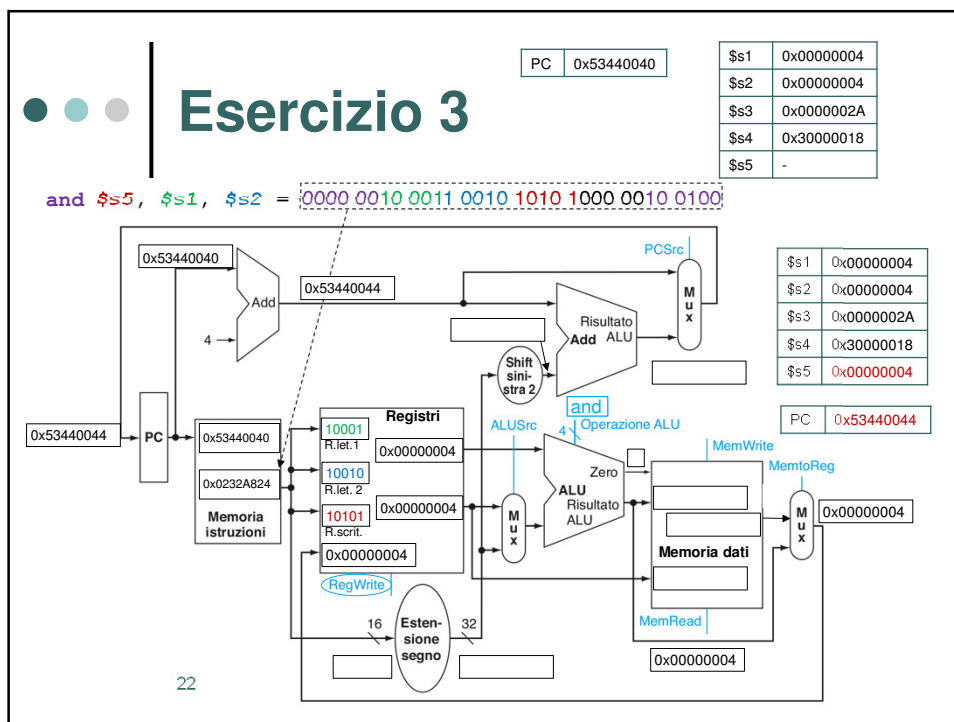


20

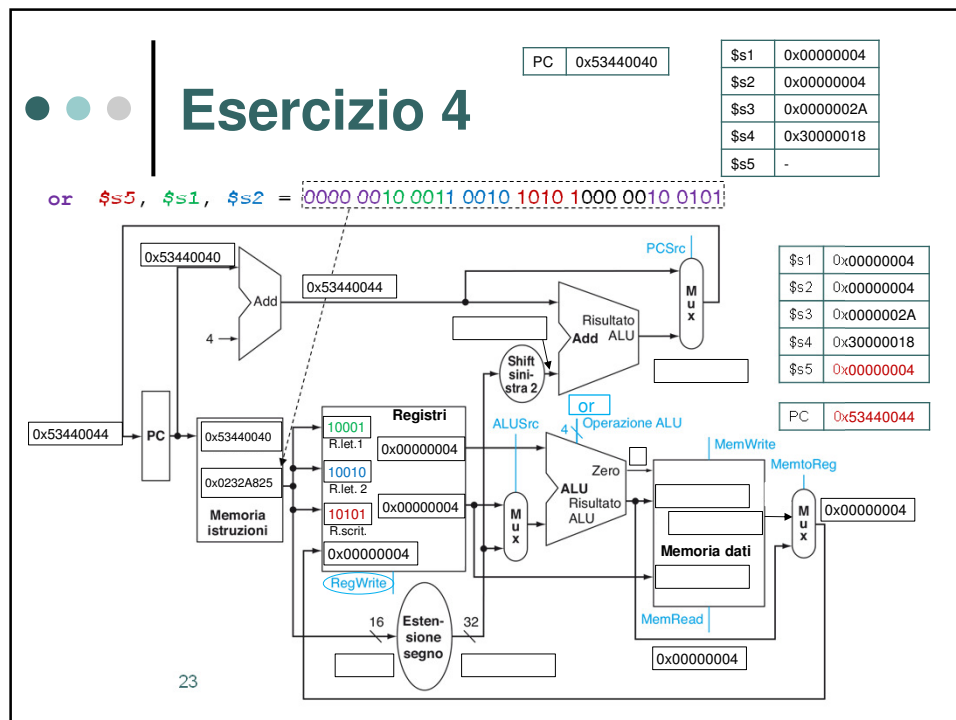
20



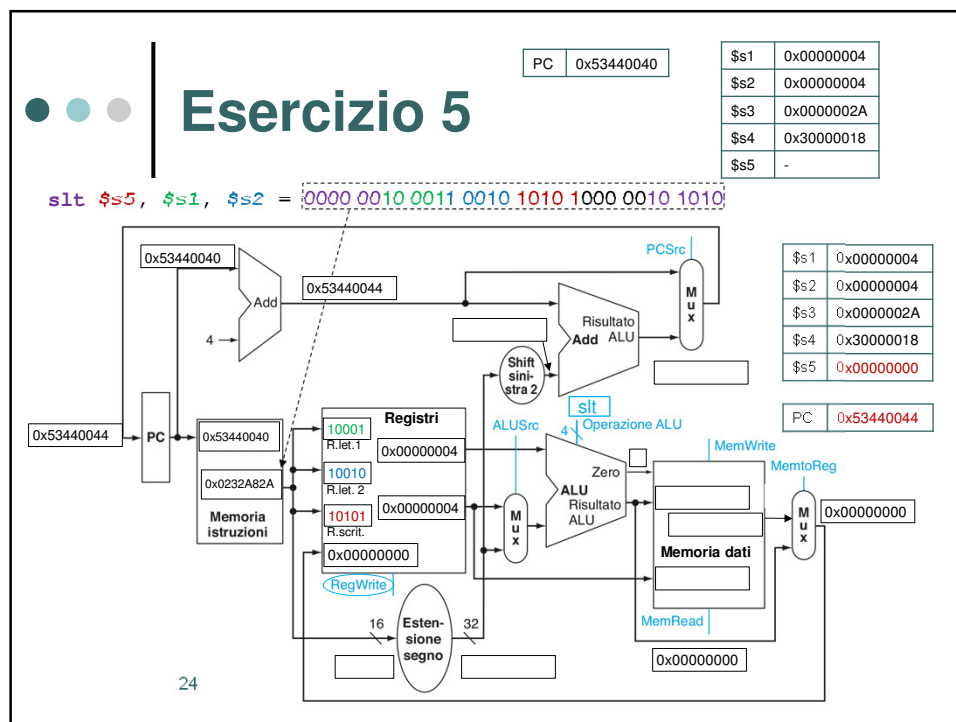
21



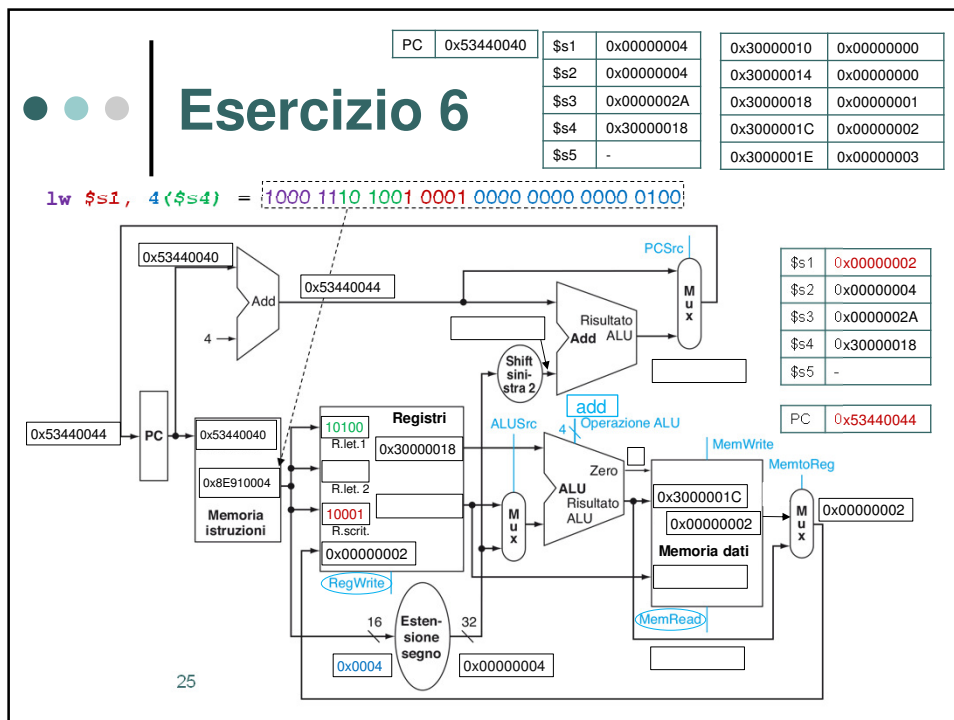
22



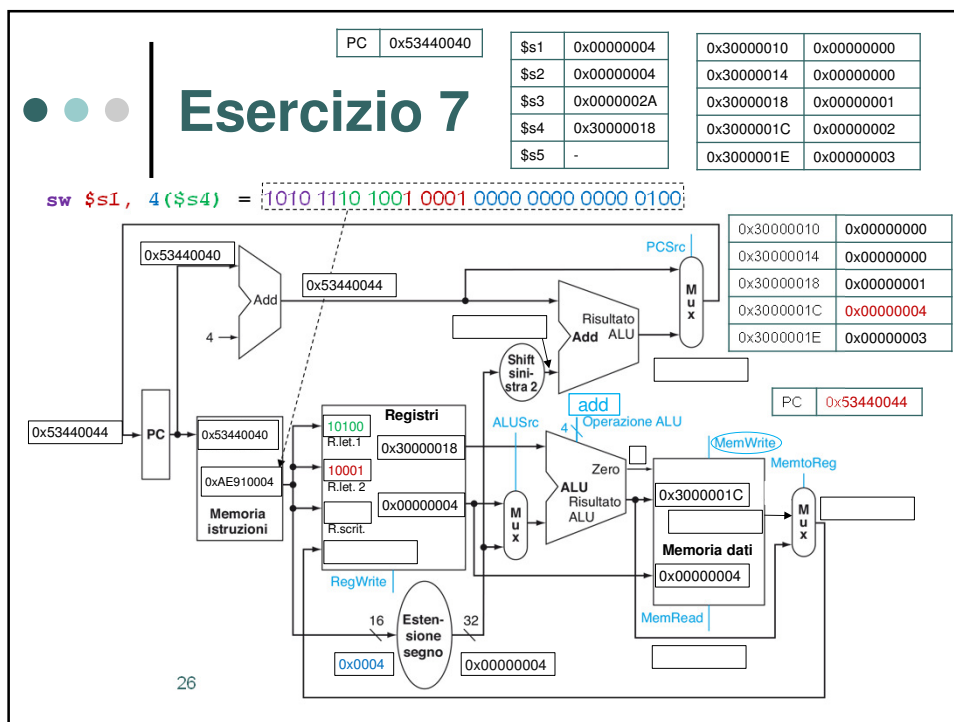
23



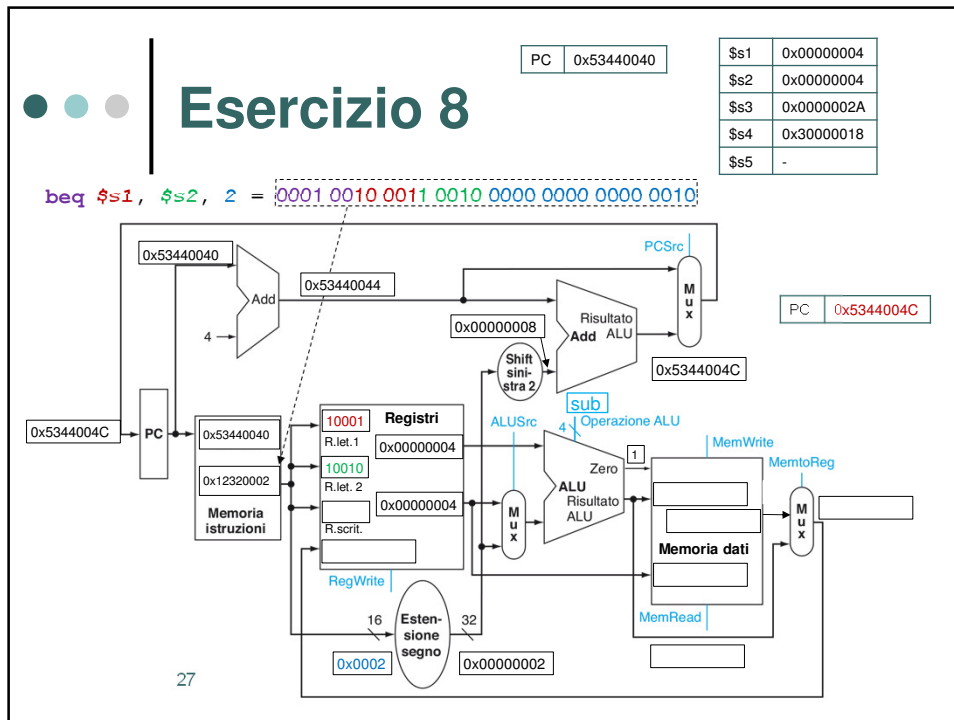
24



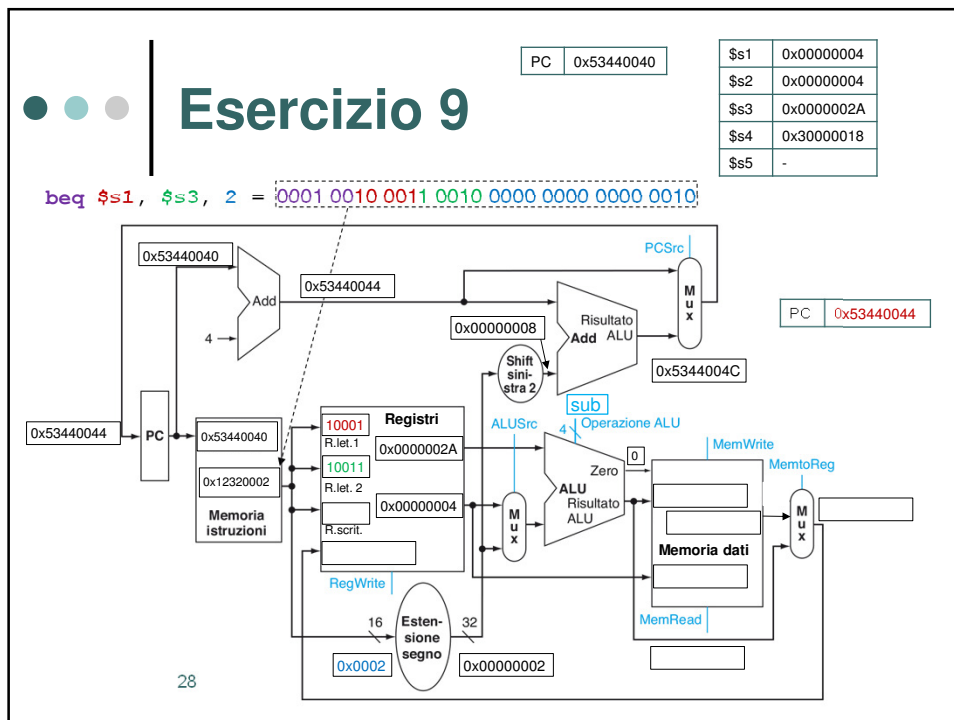
25



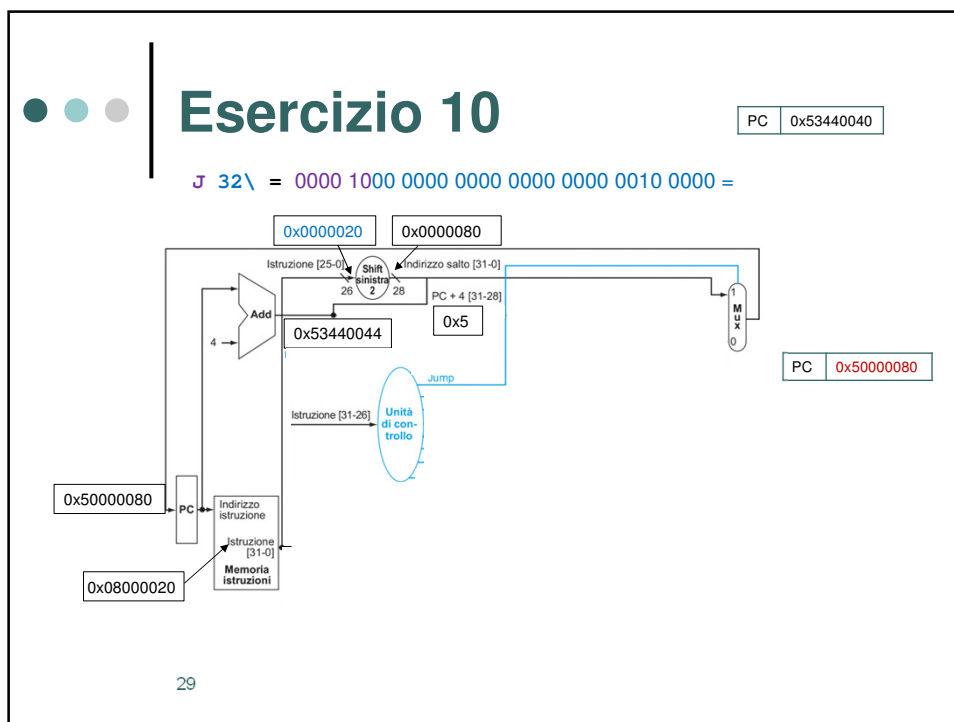
26



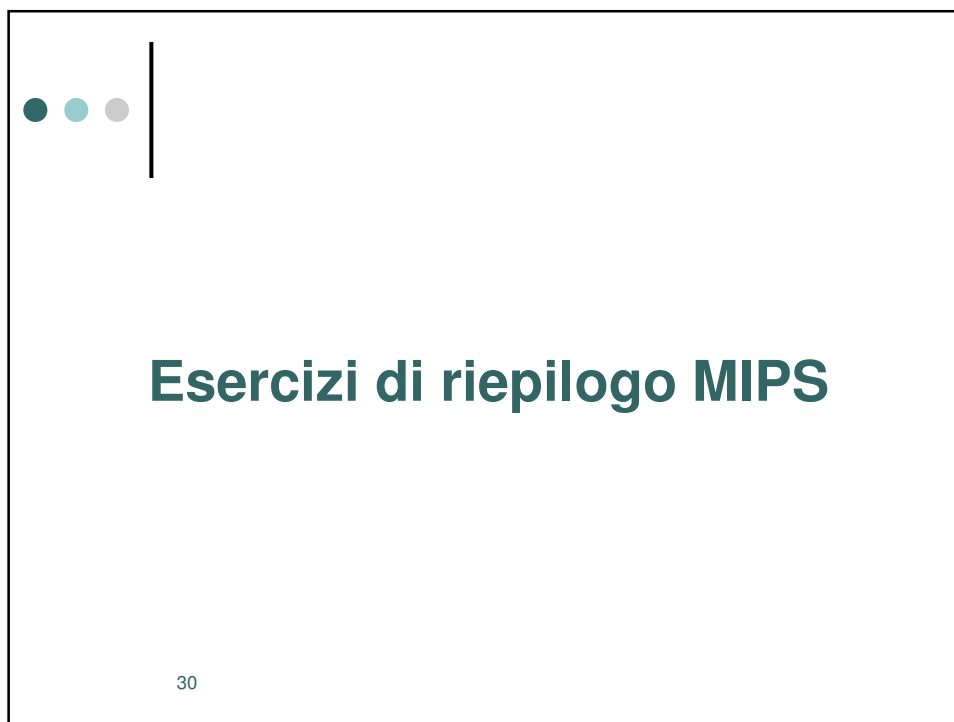
27



28



29



30



Esercizio 1

Assegnamento:

`i = 0`

Assembler (`$s0` contiene `i`)

`add $s0, $zero, $zero`

oppure

`addi $s0, $zero, 0`

31

31



Esercizio 2

Assegnamento:

`i = 5`

Assembler (`$s0` contiene `i`)

`addi $s0, $zero, 5`

32

32



Esercizio 3

Somma:

$i = j + k + h$

Assembler (\$s0 contiene i, \$s1 contiene j, \$s2 contiene k, \$s3 contiene h)

```
add $s0, $s1, $s2
```

```
add $s0, $s0, $s3
```

33

33



Esercizio 4

Assegnamento di Array con indice costante:

$i = A[3]$

Assembler (\$s0 contiene i, \$s1 contiene l'indirizzo base del vettore A)

```
lw $s0, 12($s1)
```

34

34



Esercizio 5

Assegnamento in Array con indice costante:

$A[6] = 5$

Assembler (\$s0 contiene i, \$s1 contiene l'indirizzo base del vettore A)

```
addi $t0, $zero, 5
sw   $t0, 24($s1)
```

35

35



Esercizio 6

Assegnamento di valore da un Array con indice variabile:

$i = A[j]$

Assembler (\$s0 contiene i, \$s1 contiene l'indirizzo base del vettore A, \$s2 contiene j)

```
sll $t0, $s2, 2    # j*4
add $t0, $t0, $s1  # indirizzo di A[j]
lw  $s0, 0($t0)
```

36

36



Esercizio 7

Assegnamento in un Array con indice variabile:

$A[j] = i$

Assembler (\$s0 contiene i, \$s1 contiene l'indirizzo base del vettore A, \$s2 contiene j)

```
sll $t0, $s2, 2    # j*4
add $t0, $t0, $s1  # indirizzo di A[j]
sw  $s0, 0($t0)
```

37

37



Esercizio 8

Costrutto if then else

```
if (i==j) {f = g+h}
else {f = g-h}
```

Assembler (\$s0 contiene i, \$s1 contiene j, \$s2 contiene f, \$s3 contiene g, \$s4 contiene h)

```
bne $s0, $s1, else    #salta a else
add $s2, $s3, $s4      # f=g+h
j fine                 #salta a fine
else: sub $s2, $s3, $s4 # f=g-h
fine:
```

38

38



Esercizio 9

Costrutto if con AND

```
if ((i == j) && (k == h)) {h = i + j}
```

Assembler (\$s0 contiene i, \$s1 contiene j, \$s2 contiene k, \$s3 contiene h)

```
    bne $s0, $s1, fine #se i!=j esco
    bne $s2, $s3, fine #se h!=k esco
    add $s3, $s0, $s1 # h=i+j
fine:
```

39

39



Esercizio 10

Costrutto if con OR

```
if ((i == j) || (k == h)) {h = i + j}
```

Assembler (\$s0 contiene i, \$s1 contiene j, \$s2 contiene k, \$s3 contiene h)

```
    beq $s0, $s1, then # se i==j entro
    bne $s2, $s3, fine # se h!=k esco
then: add $s3, $s0, $s1 # h=i+j
fine:
```

40

40



Esercizio 11

Costrutto for

```
g=0;
for (i=0; i<100; i++) {g=g+2;}
h=g;
```

Assembler (\$s1 contiene i, \$s2 contiene g, \$s3 contiene h)

```
      addi $s2, $zero, 0      # g=0
      addi $s1, $zero, 0      # i=0
for:   slti $t0, $s1, 100      # $t0 = 1 se i<100
                                # $t0 = 0 se i>=100
      beq  $t0, $zero, fine    # se i>=100 salta a fine
      addi $s2, $s2, 2        # g=g+2
      addi $s1, $s1, 1        # i++
      j    for                # vai a for
fine:  add  $s3, $s2, $zero    # h=g
```

41

41



Esercizio 12

Costrutto while

```
while (i<=50) {i=i-h}  //(i<= 50 è uguale a i<51)
g=i
```

Assembler (\$s1 contiene i, \$s2 contiene g, \$s3 contiene h)

```
for:   slti  $t0, $s1, 51      # $t0 = 1 se i<51
                                # $t0 = 0 se i>=51
      beq  $t0, $zero, fine    # se i>=51 salta a fine
      sub  $s1, $s1, $s3      # i=i-h
      j    for                # vai a for
fine:  addi $s2, $s1, 0        # g=i
```

42

42

Esercizio 13

Costrutto while

```
while (i>50) {i=i-h;} // (i>50 è uguale a i>=51)
g=i;
```

Assembler (\$s1 contiene i, \$s2 contiene g, \$s3 contiene h)

```
for:  slti  $t0, $s1, 51      # $t0 = 1 se i<51 (fuori)
                                # $t0 = 0 se i>=51 (for)
      bne  $t0, $zero, fine   # se i<51 salta a fine
      sub  $s1, $s1, $s3     # i=i-h
      j    for               # vai a for
fine:  addi $s2, $s1, 0       # g=i
```

43

43

Esercizio 14

Costrutto while

```
while (i>=50) {i=i-h;}
g=i;
```

Assembler (\$s1 contiene i, \$s2 contiene g, \$s3 contiene h)

```
for:  slti  $t0, $s1, 50      # $t0 = 1 se i<50
                                # $t0 = 0 se i>=50
      bne  $t0, $zero, fine   # se i<50 salta a fine
      sub  $s1, $s1, $s3     # i=i-h
      j    for               # vai a for
fine:  addi $s2, $s1, 0       # g=i
```

44

44



Esempio di esame (seconda parte)

45

45



Esercizio 1

Descrivere il ciclo fetch-execute in una CPU a ciclo unico.

Soluzione:

Il ciclo fetch-execute ha due fasi:

1. La fase di Fetch preleva l'istruzione dalla memoria.
2. La fase di Execute decodifica l'istruzione e la esegue.

Nel caso della CPU a ciclo unico:

1. La fase di Fetch utilizza il **PC** per caricare dalla **memoria istruzioni** l'istruzione corrente e usa **due sommatori** per calcolare la prossima istruzione.
2. La fase di Execute
 1. decodifica l'istruzione utilizzando l'**unità di controllo**,
 2. esegue l'istruzione utilizzando i **registri**, una **ALU** e una **memoria dati**.

46

46



Esercizio 2.1 a

Calcolo di **A[0]** (lettura in memoria)

```
lw    $t2, 0($s4)    # $t2 = A[0]
```

Calcolo di **A[i] = A[0]**

```
lw    $t2, 0($s4)    # $t2 = A[0]
```

```
sll   $t3, $s0, 2    # i*4
```

```
add   $t3, $s4, $t3  # indirizzo di A[i]
```

```
sw    $t2, 0($t3)    # A[i] = A[0]
```

47

47



Esercizio 2.1 b

for (**i=1**; **i<=3**; **i++**) {...} ovvero **i<4**
 codice modificato:

```

i=1
Loop: if (i>=4) {goto Esci;}
...
i=i+1;
goto Loop;
Esci:
  
```

```

addi $s0, $zero, 1    # i=1
Loop: slti $t0, $s0, 4    # se i>=4 allora $t0=0
beq  $t0, $zero, Esci  # se $t0=0 allora vai a Esci
...
addi $s0, $s0, 1      # i=i+1
j    Loop              # vai a Loop
Esci:
  
```

48

48



Esercizio 2.1

```
i in $s0
A in $s4
```

for (i=1; i<=3; i++) {A[i]= A[0]}

codice modificato:

```
i=1
Loop: if (i>=4) {goto Esci;}
      A[i]= A[0];
      i=i+1;
      goto Loop;
Esci:
```

```
addi $s0, $zero, 1      # i=1
Loop: slti $t0, $s0, 4    # se i>=4 allora $t0=0
      beq $t0, $zero, Esci # se $t0=0 allora vai a Esci
      lw $t2, 0($s4)      # $t2 = A[0]
      sll $t3, $s0, 2     # i*4
      add $t3, $s4, $t3   # indirizzo di A[i]
      sw $t2, 0($t3)      # A[i] = A[0]
      addi $s0, $s0, 1    # i=i+1
      j Loop              # vai a Loop
Esci:
```

49

49



Esercizio 2.2

Finito il for, la variabile `i` contiene 4 quindi in `$s0` avrò:
0x00000004

50

50

● ● ● | Esercizio 3.1

beq \$s0, \$t1, -1 è

0001 0010 0000 1001 1111 1111 1111 1111

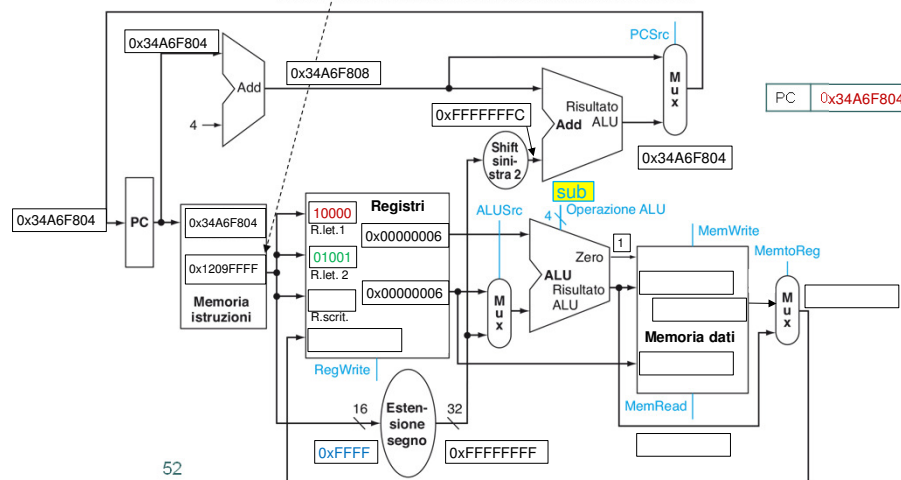
0x1209FFFF

51

51

● ● ● | Esercizio 3.2

```
beq $s0, $t1, -1 = 0001 0010 0000 1001 1111 1111 1111 1111
```



52

52